Managing the Secure Software Development

Radek Fujdiak[1,2], Petr Mlynek[1,2], Pavel Mrnustik[2], Maros Barabas[1], Petr Blazek[1], Filip Borcik[1], and Jiri Misurec[1]

[1]Brno University of Technology, Department of Telecommunications, Brno, Czech Republic
Email: {fujdiak,xblaze24,mlynek,misurec,Maros.Barabas,Filip.Borcik}@vutbr.cz
[2]Trustport, Brno, Czech republic
Email: {Pavel.Mrnustik}@trustport.com

***Abstract*—Nowadays, software development is a more complex process than ever was and it faces the challenges, where security became one of the most crucial. The security issues became an essential part of software engineers and understanding the vulnerabilities, risks and others became the everyday bread. The needs of security in software development resulted in the creation of the so-called Secure Software Development Life Cycle (SSDLC). This is a methodological concept included in classical Software Development Life-Cycle, which is described by five main phases - analysis, design, implementation (building), testing, and evaluation (deployment and maintenance). The SSDLC adds another dimension ensuring the security. We introduce our same named tool "Secure Software Development Life-cycle", which follows the general idea and goes beyond it. Our tool helps to create security, hardening, testing, and validation reporting guidelines for selected use-cases. This tool is an environment for defining the current and future security requirements based on the collection of standards, recommendations, best practice, and many others. Connecting the SSDLC with other tools improves the general level of automation of the Product Life Cycle (PLC). The SSDLC gives a connection and context among security, safety and performance parameters. Compared with static security requirements definition, the SSDLC provides simple future extension and straight integration to the PLC process with non- or nearly-non personal (human) interaction.**

***Index Terms*—Security, Software development lify cycle, Development, Software engineering, Management**

I. Introduction

The product development evolved continuously over time together with the new technological phenomena. Nowadays, the product life cycle (PLC) or product life cycle (PLC) management is a business activity of managing, in the most efficient way, company's products all the way across their life cycles; from the very first idea for a product all the way through until it is retired and disposed off [1]. One of today's most important parts of PLC processes is without doubts the software development. Over the past years, many different languages including multi-language programming [2], cloud computing [3], micro-services [4], new hybrid programming techniques [5] as well as inclusion of new phenomena such as artificial intelligence [6], machine learning [7], advanced modeling [8] and simulation [9] have changed the area of software development. The process became very complex including multiple stages, which ensures the required quality of final product [10] (see Fig. 1). Requirement analysis often contains also requirement gathering, analysis and it is focused on answering basic questions such as who will use the system, how the system will be used, what are the inputs/outputs and others [11]. Design phase includes definition of specific hardware or software requirements together with overall system architecture design [12]. Development and coding or often also called as implementation phase contains divided modules or units, which are one by one implemented to meet the requirements [13]. Testing phase is actual check whether or not the requirements are met [14]. Deployment phase starts after successful testing in previous stage and it is already delivered to the customers [15]. Operation and maintenance includes actions, which comes from real usage of the system, which was not possible to discover during the test phase [16].

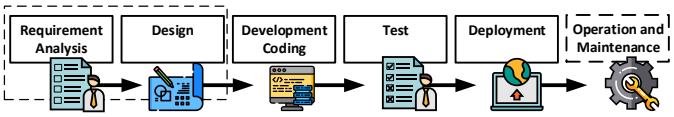

Fig. 1. Multiple stages recognized in Software Development Life Cycle (SDLC).

Nowadays, the software products serve our everyday tasks, but also ensure the functionality of the most crucial applications such as hospital systems [17], critical infrastructure [18], banking systems [19], communication and data infrastructure [20], transportation [21], and many others. Therefore, the security became one of the crucial parts of developing reliable software products [22]. Moreover, the more complex environment increases also the possibility of software bugs, mistakes, vulnerabilities, malfunctions and others [23]. In this paper, we focus on the security of SDLC together with related challenges and issues. Moreover, we introduce our tool, which helps in the software product development stages to improve cooperation and provide clear guidance on security requirements necessary for creating reliable applications.

The rest of this paper is organized as follows: Section II introduces the current state of the art for secure software development life cycle, including the different approaches and tools. Further, Section III provides description of our tool dealing with secure software development life cycle. Its main characteristic follows in Section IV, which summarizes via examples several use-cases, where our software was used and how it helps to deal with product life cycle stages. Finally, Section IV summarizes our conclusions, findings, and suggests future work.

II. State of the Art

There has been already high number of software engineering surveys as reported in [24], which summarized more than 25 survey works connected with software development worldwide as well as Turkey region in particular. The most interesting findings are: (i) Most commonly used standards for SDLC are ISO 9000 and CMMI (Capability Maturity Model Integration); and (ii) most effort is focused on development. However, a standardized approach to software engineering is still not a full warranty of reliable product. Therefore, the Secure SDLC (SSDLC) come-up and currently there are several different standards and guidance, which are applicable to it such as [28]–[32]: OWASP, ISO/IEC (such as 13335, 13849, 21434, 26262, 27002, or 27034), NIST, IEC standards (such as 61580 or 62443), and many others. The Secure SDLC has many different approaches, but we can summarize it as another layer to standard SDLC (see Fig. 2).

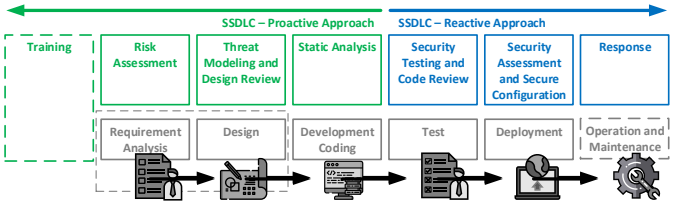

Fig. 2. Another layer of SSDLC included in the SDLC model [33]–[36].

The SSDLC adds other tasks to the already current stages of SDLC. There are two different approaches [37]–[39]: (i) Proactive approach (Preventing all possible flows and breaches at the very beginning of the project and implementing solutions in a secure way on required level) and (ii) Reactive Approach (Ensure security before release and maintain it through the product's existence). Proactive approach should be always considered as it brings several advantages over reactive approach. The highest benefit of proactive approach is finding a issues or bugs early in the SDLC processes (up to development stage) before release. This of course save valuable resources and time to fix the founded issue. Moreover, it is also much easier to correct bugs when the software is not yet implemented. Last but not least, the considered training of employees or staffs on security practices is valuable investment, which might solve many issues even before the project starts. On the other hand, the reactive approach includes additional testing due to the later stages of SDLC or even after everything has already been built (when security needs to be implemented retroactively). This approach adds another tasks to the developers and testers such as penetration testing, dynamic code analysis, vulnerability scanning, incident response plan and others. This approach adds over the proactive approach possibility to prevent all possible flows and breaches, which were not discovered or was overseen in the early stage of SDLC. However, already complex SDLC became with these even more complex SSDLC with more tasks, more decision issues, and more management load. Therefore, the management approach for SSDLC is required.

III. Managing the Secure SDLC

The management approach brings another level to the SDLC and improves it. In particular, our Management SSDLC tool (mSSDLC) brings:

- general security controls for information systems for effective risk management;
- flexible catalogue of security controls to meet the security threats, requirements and technologies;
- measurement metrics for security control effectiveness.

The most important from these points are:

- Integration of security activities into the application development process (creation of security requirements, security design analysis, guidelines for developers, code review, penetration tests and others).
- Depending on the business criticality of the developed application, the appropriate security engagement in development is made to ensure that security activities do not unnecessarily disturb the development of the application.
- Based on generally accepted standards and best practice (NIST FIPS, STIG, Microsoft SDL, CIS Benchmarks) to treat the various attack vectors with even quality.
- Ability to integrate security activities into different developmental models (especially Waterfall, Agile development).

Using the results from the interviews and documents analysis, we have developed a secure SDLC integration program, mSSDLC, including recommended policies, guidelines, and knowledge transfer. Secure software development has three elements best practices, process improvements, and metrics of software development process that focuses on these elements. The goal is to minimize security-related vulnerabilities in the design, code, and documentation and to detect and eliminate vulnerabilities as early as possible in the development life cycle. The simplified block diagram of mSSDLC is shown in Fig. 3

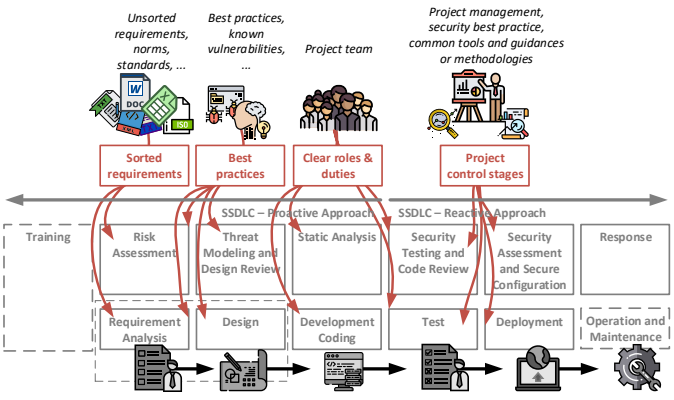

Fig. 3. The mSSDLC tool and management approach to SSDLC.

The main added values for SSDLC stages are:

- Requirements and Analysis - Over functional, non-functional and technological requirements it is also added security objectives (or requirements).
- Architecture and Design - adding security requirements (security is part of the architectural design) and creating security requirements check-list. Moreover, promoting the visibility of security - Review designs for possible security issues that means to develop mitigation of all threats. Finally, adding baseground for risk analysis and threat modelling, security requirements, privacy requirements, architecture and design review for security, design guidelines for security and threat modelling.
- Development - It helps to minimize the security issues and vulnerabilities, by adding clear roles, tasks and controls defined for unit tests and Code-review.
- Testing - During the Verification phase, the tool helps to ensure that the code meets the security and privacy tenets established in the previous phases. This is done through security and privacy testing, and a security push, which is a team-wide focus on threat model updates, code review, testing, and thorough documentation reviewing and editing. A public release privacy review is also completed during the Verification phase.
- Deployment - After implementation, there is supposed to be a verification phase for the real devices. Security audit is an expensive approach and should be made after the final tests. This might be done by fast security evaluation based on the quantitative analysis of the final product, which should be provided by the TP methodology.

Last but not least, the main advantages to the general SSDLC approach, which the tool offers, are:

- Identifying Security Objectives - To understand key security objectives and scenarios within the SDLC stages.
- Security Design Guidelines - Create guidelines, specify security requirements, architecture and design review for security purposes.
- Threat Models - Identifying threats, attacks and vulnerabilities for specified information system, with countermeasures.
- Assessment - Penetration tests, reviews and configuration audits; simulation of attacks and scenarios to uncover weaknesses and vulnerabilities during the development and in deployment.

IV. Application areas

The mSSDLC helps in many different areas with giving the management context to SSDLC. We are bringing one particular use-case, where mSSDLC was used with a lesson learned. The use-case might be described as follows: Early requirements/design PLC stage of a critical control system for asynchronous motor, where the most important parameter was the performance (i.e., fast stop command delivery etc.). However, the cyber-security standards IEC 62443 brought the security questions and new requirements, which must be faced in the PLC stages. The gathered security requirements from the specification and best practice were included in the mSSDLC and used for another tasks of SSDLC such as final minimum security requirements for design phase, trade-off analysis and decision process. The simplified block-diagram is shown in Fig. 4.

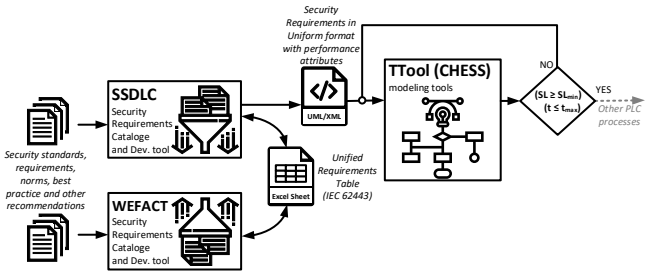

Fig. 4. Simplified block-diagram for selected use-case.

The WEFACT[1] was a static database in the project, which was automatically actualized with defined requirements from SSDLC and validated. Moreover, the uniform UML/XML format was used as an input in the simulation and modeling tools - TTool[2] and CHESS[3]. The minimal security requirement provided from mSSDLC was used to simulate the impact on the performance (particularly to get the delay growth). Thanks to the early-stage analysis via mSSDLC and management approach the early-stage gets sufficient information for pre-simulations, which gave enough valid information for the decision making process and saved time as well as costs. Without the mSSDLC, the whole early-stage pre-simulation would not be possible and the right decision would be very hard to make with high probability to make wrong decision, which would increase future costs and time for application development.

V. Conclusion

We bring the very current state of the art for SDLC (Software Development Life Cycle) as well as for SSDLC (Secure Software Development Life Cycle). We introduce both approaches, highlight the challenges and bring-up the new topic of management approach in the SSDLC together with introduction of our mSSDLC tool. The whole development is unfortunately not anymore just an engineering work. The project management plays crucial role and helps to move forward in the right direction through the development life-cycle. As mentioned, there are different teams involved with many decisions, which need to be made thought the whole SDLC, before the product hits the market [25]. The early stage of SDLC is about creating requirements, which must be fulfilled by the final product and even at this early stage we must fight the different areas of requirements, which

[1]WEFACT is Workflow Engine for Analysis, Certification and Test, which is commonly used in the industrial use-cases.

[2]TTool is SysML modeling tool, which implements Security and Performance analysis for many different applications.

[3]CHESS is Composition with Guarantees for High-integrity Embedded Software Components Assembly and it provides a model-driven, component-based methodology.

often ends in performance versus security trade-off [27]. However, the industrial applications, medical applications, airspace applications, transportation applications include another dimension of the requirements - safety, which must be also taken into account, and which makes the SDLC even more complex [26]. In this paper, we introduce the two-dimensional approach, where performance is not anymore the only issue, but also how the security requirements enter the early stage of SSDLC. However, the issue is much more complex and it is unfortunately beyond the scope of this paper. Therefore, this is also a discovered challenge, which should be focused on in the future research.

VI. Acknowledgement

This article has received funding within the National Sustainability Program under grant LO1401 and Technology Agency of the Czech Republic under grant no. TJ01000381. Our research and the idea of the paper is coming from the research conducted and supported by research project Aggregated Quality Assurance for Systems (AQUAS H2020-EU.2.1.1.7 ID: 737475). For the research, the infrastructure of the SIX Center was used.

References

[1] STARK, John. Product life cycle management. In: Product life cycle management (Volume 1). Springer, Cham, 2015. p. 1-29.

[2] MAYER, Philip; KIRSCH, Michael; LE, Minh Anh. On multi-language software development, cross-language links and accompanying tools: a survey of professional software developers. Journal of Software Engineering Research and Development, 2017, 5.1: 1.

[3] JONAS, Eric, et al. Cloud Programming Simplified: A Berkeley View on Serverless Computing. arXiv preprint arXiv:1902.03383, 2019.

[4] BRENNER, Stefan, et al. Secure cloud micro services using intel SGX. In: IFIP International Conference on Distributed Applications and Interoperable Systems. Springer, Cham, 2017. p. 177-191.

[5] WEINBERG, Volker. New PATC Course: Introduction to hybrid programming in HPC. 2017.

[6] RUSSELL, Stuart J.; NORVIG, Peter. Artificial intelligence: a modern approach. Malaysia; Pearson Education Limited,, 2016.

[7] ABADI, Martn, et al. Tensorflow: A system for large-scale machine learning. In: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16). 2016. p. 265-283.

[8] BYRNE, Barbara M. Structural equation modeling with AMOS: Basic concepts, applications, and programming. Routledge, 2016.

[9] LEWIS, P. W. A.; MCKENZIE, Ed. Simulation Methodology for Statisticians, Operations Analysts, and Engineers (1988). Chapman and Hall/CRC, 2017.

[10] STARK, John. Product lifecycle management. In: Product lifecycle management (Volume 1). Springer, Cham, 2015. p. 1-29.

[11] NURMULIANI, Nur; ZOWGHI, Didar; POWELL, S. Analysis of requirements volatility during software development life cycle. In: 2004 Australian Software Engineering Conference. Proceedings. IEEE, 2004. p. 28-37.

[12] BRUEGGE, Bernd; DUTOIT, Allen H. Object-Oriented Software Engineering. Using UML, Patterns, and Java. Learning, 2009, 5.6: 7.

[13] LEAU, Yu Beng, et al. Software development life cycle AGILE vs traditional approaches. In: International Conference on Information and Network Technology. 2012. p. 162-167.

[14] BAYER, Joachim, et al. PuLSE: A Methodology to Develop Software Product Lines. SSR, 1999, 99: 122-131.

[15] DUSTIN, Elfriede. Effective Software Testing: 50 Ways to Improve Your Software Testing. Addison-Wesley Longman Publishing Co., Inc., 2002.

[16] BENNETT, Keith H.; RAJLICH, Vclav T. Software maintenance and evolution: a roadmap. In: Proceedings of the Conference on the Future of Software Engineering. ACM, 2000. p. 73-87.

[17] KASSAB, Mohamad; DEFRANCO, Joanna; LAPLANTE, Phillip A. Software development for medical devices: State of practice. In: 2017 IEEE 28th Annual Software Technology Conference (STC). IEEE, 2017. p. 1-6.

[18] WAGNER, Christian, et al. Impact of critical infrastructure requirements on service migration guidelines to the cloud. In: 2015 3rd International Conference on Future Internet of Things and Cloud. IEEE, 2015. p. 1-8.

[19] GUMUSSOY, Cigdem Altin. Usability guideline for banking software design. Computers in Human Behavior, 2016, 62: 277-285.

[20] CUI, Laizhong; YU, F. Richard; YAN, Qiao. When big data meets software-defined networking: SDN for big data and big data for SDN. IEEE network, 2016, 30.1: 58-65.

[21] BILGIN, Berker, et al. Making the case for electrified transportation. IEEE Transactions on Transportation Electrification, 2015, 1.1: 4-17.

[22] KHAN, Muhammad Umair Ahmed; ZULKERNINE, Mohammad. Quantifying security in secure software development phases. In: 2008 32nd Annual IEEE International Computer Software and Applications Conference. IEEE, 2008. p. 955-960.

[23] RATLIFF, Zachary B., et al. The relationship between software bug type and number of factors involved in failures. In: 2016 IEEE international symposium on software reliability engineering workshops (ISSREW). IEEE, 2016. p. 119-124.

[24] GAROUSI, Vahid, et al. A survey of software engineering practices in Turkey. Journal of Systems and Software, 2015, 108: 148-177.

[25] MATHARU, Gurpreet Singh, et al. Empirical study of agile software development methodologies: A comparative analysis. ACM SIGSOFT Software Engineering Notes, 2015, 40.1: 1-6.

[26] FUJDIAK, Radek, et al. Seeking the Relation between Performance and Security in Modern Systems: Metrics and Measures. In 41st International Conference on Telecommunications and Signal Processing (TSP). International Conference on Telecommunications and Signal Processing (TSP). 2018. p. 288-293. ISBN: 978-1-5386-4695-3. ISSN: 1805-5435.

[27] ZO, Hangjung; NAZARETH, Derek L.; JAIN, Hemant K. Security and performance in service-oriented applications: Trading off competing objectives. Decision Support Systems, 2010, 50.1: 336-346.

[28] FUTCHER, Lynn; VON SOLMS, Rossouw. Guidelines for secure software development. In: Proceedings of the 2008 annual research conference of the South African Institute of Computer Scientists and Information Technologists on IT research in developing countries: riding the wave of technology. ACM, 2008. p. 56-65.

[29] HOWARD, Michael; LIPNER, Steve. The security development life cycle. Redmond: Microsoft Press, 2006.

[30] GILLIAM, David P., et al. Software security checklist for the software life cycle. In: WET ICE 2003. Proceedings. Twelfth IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises, 2003. IEEE, 2003. p. 243-248.

[31] DAUD, Malik Imran. Secure software development model: A guide for secure software life cycle. In: Proceedings of the international MultiConference of Engineers and Computer Scientists. 2010. p. 17-19.

[32] APVRILLE, Axelle; POURZANDI, Makan. Secure software development by example. IEEE Security & Privacy, 2005, 3.4: 10-17.

[33] JONES, Russell L.; RASTOGI, Abhinav. Secure coding: building security into the software development life cycle. Information Systems Security, 2004, 13.5: 29-39.

[34] DAWSON, Maurice, et al. Integrating software assurance into the software development life cycle (SDLC). Journal of Information Systems Technology and Planning, 2010, 3.6: 49-53.

[35] DAVIS, Noopur. Secure software development life cycle processes: A technology scouting report. Carnegie-Mellon Univ Pittsburgh Pa Software Engineering Inst, 2005.

[36] BARABANOV, Alexander, et al. Synthesis of secure software development controls. In: Proceedings of the 8th International Conference on Security of Information and Networks. ACM, 2015. p. 93-97.

[37] SAHU, Kavita; SHREE, R.; KUMAR, R. Risk management perspective in SDLC. International Journal of Advanced Research in Computer Science and Software Engineering, 2014, 4.3.

[38] MAHESHWARI, V.; PRASANNA, M. Integrating risk assessment and threat modeling within SDLC process. In: 2016 International Conference on Inventive Computation Technologies (ICICT). IEEE, 2016. p. 1-5.

[39] ARORA, Yojna. Approaches for Enhancing Reliability of Software Product. International Journal of Computer Applications, 2017, 161.5.